

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ФАКУЛЬТЕТ ПРИКЛАДНОЙ МАТЕМАТИКИ И ИНФОРМАТИКИ

ОТЧЕТ по лабораторной работе №2
Решение проблемы собственных значений

Выполнил:
Студент 3 курса, 7 группы
Стаселько Илья Юрьевич

Преподаватель:
Репников Василий Иванович

Минск 2025

1 Постановка задачи

1. При заданном $n \geq 10$ сгенерировать случайным образом квадратную матрицу размера $n \times n$. Для корректного применения метода вращений матрица должна быть симметричной ($A = A^T$).
2. Используя **Метод вращений (Якоби)**, решить полную проблему собственных значений (найти все собственные числа λ_i и соответствующие собственные векторы).
3. Используя **Степенной метод**, найти наибольшее по модулю собственное значение λ_{max} и соответствующий ему собственный вектор.
4. Выполнить проверку полученных результатов путем вычисления нормы невязки $\|Ax - \lambda x\|$.

2 Теоретические сведения

Проблема собственных значений заключается в нахождении чисел λ и ненулевых векторов x , удовлетворяющих уравнению $Ax = \lambda x$.

2.1 Итерационный метод вращений (Якоби)

Метод Якоби применяется для решения полной проблемы собственных значений для вещественных симметричных матриц. Любая симметричная матрица может быть приведена к диагональному виду посредством ортогонального преобразования $A = U\Lambda U^{-1}$ (спектральное разложение), где Λ — диагональная матрица с собственными числами, а столбцы U — собственные векторы.

Метод строит последовательность матриц $A_0 = A, A_1, A_2, \dots$, сходящихся к диагональной матрице. Каждая следующая матрица получается из предыдущей преобразованием подобия:

$$A_{k+1} = T_{ij}^T A_k T_{ij},$$

где $T_{ij}(\phi)$ — матрица вращения (Гивенса). Это единичная матрица, у которой элементы на пересечении i -х и j -х строк и столбцов заменены на:

$$t_{ii} = \cos \phi, \quad t_{jj} = \cos \phi, \quad t_{ij} = -\sin \phi, \quad t_{ji} = \sin \phi$$

Угол поворота ϕ выбирается так, чтобы обнулить максимальный по модулю внедиагональный элемент $a_{ij}^{(k)}$ ($i \neq j$). Формула для угла:

$$\operatorname{tg}(2\phi) = \frac{2a_{ij}}{a_{ii} - a_{jj}}, \quad |\phi| \leq \frac{\pi}{4}. \quad (1)$$

Если $a_{ii} = a_{jj}$, то $\phi = \pi/4$.

Элементы матрицы пересчитываются следующим образом:

- $a_{ii}^{k+1} = a_{ii}^k \cos^2 \phi + 2a_{ij}^k \cos \phi \sin \phi + a_{jj}^k \sin^2 \phi$
- $a_{jj}^{k+1} = a_{ii}^k \sin^2 \phi - 2a_{ij}^k \cos \phi \sin \phi + a_{jj}^k \cos^2 \phi$
- $a_{ij}^{k+1} = a_{ji}^{k+1} = 0$ (теоретически)
- Для остальных индексов m : $a_{im}^{k+1} = a_{im}^k \cos \phi + a_{jm}^k \sin \phi$.

Сходимость метода определяется величиной $t(A) = \sum_{i \neq j} |a_{ij}|^2$. При каждом шаге эта величина уменьшается, то есть $t(A_{k+1}) < t(A_k)$. Метод останавливается, когда все внедиагональные элементы становятся по модулю меньше ε .

2.2 Степенной метод

Степенной метод предназначен для нахождения наибольшего по модулю собственного значения λ_1 . Пусть $|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_n|$. Строится итерационная последовательность векторов:

$$y^k = Ay^{k-1} = A^k y^0$$

Начальный вектор y^0 раскладывается по базису из собственных векторов: $y^0 = \sum c_i x_i$. Тогда $y^k = \sum c_i \lambda_i^k x_i = \lambda_1^k (c_1 x_1 + \sum_{i=2}^n c_i (\frac{\lambda_i}{\lambda_1})^k x_i)$. При $k \rightarrow \infty$ отношение $(\lambda_i/\lambda_1)^k \rightarrow 0$, и вектор y^k стремится к направлению собственного вектора x_1 .

Собственное число вычисляется как отношение компонент или через отношение Релея (для симметричных матриц):

$$\lambda \approx \frac{(y^{k+1}, y^k)}{(y^k, y^k)}$$

Для предотвращения переполнения вектор нормируется на каждом шаге. Скорость сходимости зависит от отношения $|\lambda_2|/|\lambda_1|$.

3 Реализация алгоритма

Программа реализована на языке Python с использованием библиотеки NumPy.

3.1 Генерация матрицы

Согласно заданию, для корректной работы метода Якоби матрица должна быть симметричной.

```
1 def generate_symmetric_matrix(n):
2     A = np.random.rand(n, n) * 10
3     # Делаем симметричной: A = (A + A.T) / 2
4     A_sym = (A + A.T) / 2
5     return A_sym
```

Листинг 1: Генерация симметричной матрицы

Пояснение: Создается случайная матрица A , затем вычисляется полусумма A и её транспонированной копии. Это гарантирует выполнение условия $a_{ij} = a_{ji}$, необходимого для теоретической сходимости метода Якоби (спектральное разложение).

3.2 Реализация метода вращений (Якоби)

```
1 def get_max_off_diagonal(A):
2     # Поиск индексов p, q для max/a_pq/
3     ...
4     return max_val, p, q
5
6 # Внутри цикла метода Якоби:
7 if abs(A_curr[p, p] - A_curr[q, q]) < 1e-15:
8     phi = math.pi / 4
9 else:
10     # Формула tg(2phi) = 2*a_pq / (a_pp - a_qq)
11     phi = 0.5 * math.atan(2 * A_curr[p, q] /
12                          (A_curr[p, p] - A_curr[q, q]))
13 c = math.cos(phi)
14 s = math.sin(phi)
```

Листинг 2: Поиск элемента и вычисление угла

Пояснение: Этот блок реализует стратегию выбора "аннулируемого" элемента. Мы ищем максимальный по модулю внедиагональный элемент (стратегия максимального элемента), что обеспечивает наиболее быструю сходимость. Угол ϕ вычисляется строго по формуле (1) из теоретической части.

```

1 # Обновление диагональных элементов и a_pq
2 A_next[p, p] = c**2 * A_curr[p, p] + 2*c*s * A_curr[p, q] + s**2 * A_curr[q,
    q]
3 A_next[q, q] = s**2 * A_curr[p, p] - 2*c*s * A_curr[p, q] + c**2 * A_curr[q,
    q]
4 A_next[p, q] = 0; A_next[q, p] = 0
5
6 # Обновление остальных элементов (формулы поворота)
7 for i in range(n):
8     if i != p and i != q:
9         A_next[i, p] = c * A_curr[i, p] + s * A_curr[i, q]
10        A_next[p, i] = A_next[i, p] # Симметрия
11        A_next[i, q] = -s * A_curr[i, p] + c * A_curr[i, q]
12        A_next[q, i] = A_next[i, q] # Симметрия

```

Листинг 3: Пересчет матрицы (Вращение)

Пояснение: Данный блок выполняет преобразование $A_{k+1} = T^T A_k T$. Реализованы формулы пересчета элементов, приведенные в теории. Важно отметить, что пересчитываются только элементы в строках и столбцах с индексами p и q , остальные элементы матрицы остаются без изменений, что экономит вычислительные ресурсы ($O(n)$ операций на итерацию вместо $O(n^2)$).

```

1 # V_new = V_old * G
2 V_next = V.copy()
3 for i in range(n):
4     V_next[i, p] = c * V[i, p] + s * V[i, q]
5     V_next[i, q] = -s * V[i, p] + c * V[i, q]
6 V = V_next

```

Листинг 4: Накопление собственных векторов

Пояснение: Согласно теории, матрица собственных векторов U равна произведению всех матриц вращения. Здесь мы итеративно умножаем текущую матрицу V на матрицу вращения справа.

3.3 Реализация Степенного метода

```

1 def power_iteration_method(A, tol=1e-7, max_iter=1000):
2     x = np.random.rand(n)
3     x = x / np.linalg.norm(x) # Нормировка
4
5     for k in range(max_iter):
6         x_next = np.dot(A, x) # x = A * x
7         lam = np.dot(x_next, x) # Отношение Релея (т.к. x нормирован)
8
9         norm = np.linalg.norm(x_next)
10        x_next = x_next / norm # Нормировка
11
12        if abs(lam - lam_prev) < tol: # Проверка сходимости
13            return lam, x_next, k + 1
14        x = x_next

```

Листинг 5: Степенной метод

Пояснение: Реализуется итерационный процесс $x^{(k)} = Ax^{(k-1)}$. Нормировка вектора на каждом шаге необходима для избежания переполнения (если $|\lambda| > 1$) или зануления (если $|\lambda| < 1$). Собственное число оценивается через скалярное произведение (Ax, x) , что для симметричных матриц эквивалентно отношению Релея и дает квадратичную сходимость оценки собственного числа.

4 Результаты работы программы

Была сгенерирована симметричная матрица 10×10 . Ниже приведен полный вывод программы.

Сгенерированная матрица A :

3.12225	4.36959	4.62264	6.30508	1.83795	7.45329	3.70525	3.02334	6.12656	4.92503
4.36959	0.64630	4.07331	3.31129	4.21876	6.63647	2.98948	6.73838	5.90995	6.56329
4.62264	4.07331	9.25148	3.11817	3.81008	5.19722	5.55225	6.83216	4.49920	4.61396
6.30508	3.31129	3.11817	5.37658	5.18686	5.01110	6.44885	5.67737	9.06338	6.69095
1.83795	4.21876	3.81008	5.18686	6.17132	5.05822	3.34064	4.91201	7.50255	3.30261
7.45329	6.63647	5.19722	5.01110	5.05822	4.65063	4.91212	9.00260	3.63564	3.48102
3.70525	2.98948	5.55225	6.44885	3.34064	4.91212	6.09537	5.18386	7.46078	3.86125
3.02334	6.73838	6.83216	5.67737	4.91201	9.00260	5.18386	3.16402	0.23577	6.58718
6.12656	5.90995	4.49920	9.06338	7.50255	3.63564	7.46078	0.23577	4.59109	5.29423
4.92503	6.56329	4.61396	6.69095	3.30261	3.48102	3.86125	6.58718	5.29423	3.30881

4.1 Результаты Метода вращений (Якоби)

- Количество итераций: 154
- Найденные собственные числа:

[50.57080214 8.84558894 4.62403891 3.87319235 1.49468676 1.13495721
-1.76012347 -5.2339818 -7.52125809 -9.65006171]

- Проверка для макс. с.ч. (Якоби):

$$\|Ax - \lambda x\| = 3.06513 \cdot 10^{-13}$$

4.2 Результаты Степенного метода

- Количество итераций: 10
- Найденное максимальное по модулю с.з.: 50.57080214
- Проверка (Power):

$$\|Ax - \lambda x\| = 8.41044 \cdot 10^{-7}$$

4.3 Сравнение результатов

- Разница между с.з. Якоби и Степенного метода: $3.12639 \cdot 10^{-13}$
- Библиотечное значение (numpy.linalg.eigvalsh): 50.57080214

5 Выводы и анализ результатов

1. Сравнение методов: **Метод Якоби** позволил найти *весь спектр* матрицы (все 10 собственных значений). Он показал чрезвычайно высокую точность (невязка порядка 10^{-13}), что близко к машинной точности. Однако для этого потребовалось 154 итерации, что объясняется необходимостью обнулять внедиагональные элементы многократно (обнуление одного элемента может "портить" уже обнуленные ранее). **Степенной метод** позволил найти только *одно* (максимальное по модулю) собственное число. Он сошелся очень быстро (всего за 10 итераций). Это связано с тем, что доминирующее собственное число ($\lambda_1 \approx 50.57$) значительно превосходит по модулю следующее ($\lambda_{10} \approx -9.65$ или $\lambda_2 \approx 8.84$). Хорошая разделенность спектра обеспечивает быструю сходимость степенного метода.

2. Точность: Степенной метод показал меньшую точность (10^{-7}) по сравнению с методом Якоби. Это связано с тем, что критерий остановки в степенном методе (разность последовательных приближений λ) является менее строгим условием, чем обнуление всех внедиагональных элементов в методе Якоби. Тем не менее, значение 50.57080214 совпало у обоих методов и с библиотечным значением.

3. Итог: Для полной проблемы собственных значений симметричных матриц метод Якоби является надежным и точным инструментом. Для поиска только спектрального радиуса матрицы степенной метод является значительно более эффективным вычислительно.